

ADVENTIST UNIVERSITY OF CENTRAL AFRICA

School of Postgraduate Studies

Master's in Big Data Analytics

Real-Time eCommerce Purchase Prediction with HDFS, Kafka, PySpark, MLlib and Django

*An End-to-End Big Data Pipeline for Streaming
Clickstream Analytics and Predictive Modelling*

Big Data Essentials — Final Course Project

Submitted by

Ahourdet Donambi Thierry

Emmanuel Manzi

Gwiza Rodrigue

Niyindora Emile

SHEMA Joshua

Course Instructor

Dr. Kundan Kumar

July 2026

Contents

1	Project Overview	3
1.1	Objective	3
1.2	Case study and dataset	3
1.3	Outcome summary	3
2	System Architecture	4
2.1	Overall design	4
2.2	Component responsibilities	5
2.3	Storage strategy	5
2.4	Database schema	6
3	Implementation	7
3.1	Distributed storage — HDFS	7
3.2	Message transport — Kafka	8
3.3	Kafka producer — ingestion job	8
3.4	Kafka consumer — PySpark Structured Streaming	9
3.5	Feature engineering	10
3.5.1	Leakage control	10
3.5.2	Train/serve feature parity	11
3.6	Prediction modelling with MLlib	11
3.6.1	Model selection	11
3.6.2	Chronological splitting	11
3.6.3	Class imbalance	12
3.6.4	Decision threshold tuning	12
3.7	Champion/Challenger retraining	12
3.8	Django dashboard	12
4	Technologies and Version Information	15
5	Dependencies	15
5.1	Spark JAR	15
5.2	Python packages by service	16
5.3	Resource allocation	16
6	Testing	17
7	Results and Findings	18
7.1	Dataset after session aggregation	18
7.2	Threshold sweep on the validation set	18
7.3	Held-out test performance	19
7.4	Findings	19
7.5	Limitations	20
8	Deployment and Cloud Strategy	21
8.1	Local deployment sequence	21
8.2	AWS RDS migration	21
9	Conclusion	22
A	Training Run Log	23
B	Model Registry Verification	23

C Source Code Repository**23**

1 Project Overview

1.1 Objective

This project implements an end-to-end big data system that predicts, from a live clickstream of browsing events on an online store, whether a customer session will end in a purchase. The prediction is produced from behaviour observed *during* the session — views, cart additions, removals, product and category breadth, price range and dwell time — and is surfaced on a live web dashboard alongside the operational health of the model serving it.

The work satisfies the four objectives set out in the course brief:

1. **Real-time data processing.** Events are streamed through Apache Kafka and consumed by PySpark Structured Streaming.
2. **Prediction modelling.** A Spark MLlib classification pipeline is trained on historical data held in HDFS and applied to the live stream.
3. **Database integration.** Processed predictions, model metadata and pre-aggregated counters are persisted to MySQL.
4. **Web dashboard visualisation.** A Django application renders live predictions, model registry history and streaming trends.

1.2 Case study and dataset

The chosen case study is **customer behaviour prediction** on the Kaggle *eCommerce Behavior Data from Multi-Category Store* dataset. Each row is a single user interaction with a product, carrying an event timestamp, an event type (`view`, `cart`, `remove_from_cart`, `purchase`), product and category identifiers, brand, price, user identifier and session identifier.

Two monthly files are used, and the split between them is what makes the system a genuine streaming pipeline rather than a batch job with a web front end:

File	Approx. size	Role	Path in HDFS
2019-Oct.csv	5.3 GB	Historical training corpus	/data/raw/2019-10
2019-Nov.csv	9.0 GB	Replayed as the live stream	/data/raw/2019-11

Table 1: Dataset partitioning. Both files exceed the 1 GB minimum required by the project brief.

October is never streamed; it is the offline corpus the first model learns from. November is never read directly by the trainer; it arrives through Kafka one event at a time, exactly as production traffic would, and only reaches the training job later via the Parquet archive the streaming job writes.

1.3 Outcome summary

A logistic regression model trained on 9.4 million October sessions achieves an **AUC of 0.828** on a chronologically held-out test set, recovering 46.2% of purchasing sessions at 47.9% precision against a 6.7% base rate — roughly a seven-fold lift. The full pipeline runs as six containerised services orchestrated by Docker Compose, with automated retraining and zero-downtime model promotion.

2 System Architecture

2.1 Overall design

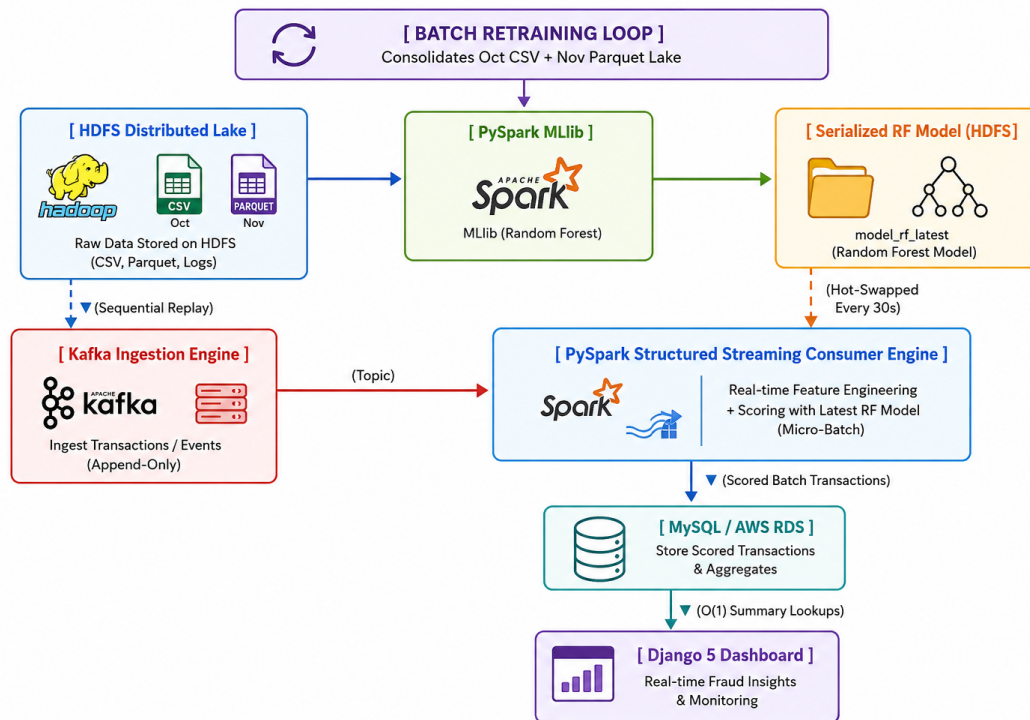


Figure 1: End-to-end system architecture: HDFS as the data lake, Kafka as the transport, PySpark for streaming and batch compute, MySQL as the serving store, and Django as the presentation layer.

The system is built around **two independent loops** operating over the same Kafka topic at very different timescales.

Loop A — real-time scoring (seconds). The ingestion job reads November events out of HDFS over WebHDFS, line by line, and publishes them to the Kafka topic `clickstream-events`. The streaming job consumes that topic and fans out into two sinks: one archives every raw event to HDFS as Parquet, the other maintains incremental per-session feature aggregates, scores them with the currently active model, and writes the results to MySQL. The Django dashboard reads MySQL.

Loop B — batch retraining (scheduled). The training job reads the October baseline from HDFS *and* whatever November data the streaming archive has accumulated, merges them, engineers session-level features, trains a new model, and evaluates it against the incumbent. The model artifact is written to HDFS; only metadata and metrics go to MySQL.

The coupling point. The two loops never call one another. They are joined solely through the `model_registry` table in MySQL. The training job writes a row and may flip `is_active`; the streaming job polls that table every 30 seconds and reloads the model from HDFS only if the active path has changed. This is what allows a newly promoted model to take over live scoring without restarting any container.

2.2 Component responsibilities

Service	Technology	Responsibility
namenode datanode	/ Hadoop HDFS 3.2.1	Data lake. Stores raw CSV, Parquet archives, serialised model artifacts and Spark streaming checkpoints.
kafka	Kafka 3.7.0 (KRaft)	Message transport. Single topic, three partitions, keyed by <code>user_session</code> .
ingestion-job	Python + WebHDFS + confluent-kafka	Kafka producer. Replays historical rows from HDFS into the topic at a controlled rate.
streaming-job	PySpark Structured Streaming 3.5.1	Kafka consumer. Archives raw events to Parquet, aggregates sessions, applies the Champion model, writes predictions to MySQL.
training-job	PySpark + MLlib 3.5.1	Batch trainer. Feature engineering, chronological split, model fitting, threshold tuning, Champion/Challenger evaluation, registry updates.
mysql	MySQL 8.0	Serving store. Model registry, prediction rows, pre-aggregated counters.
django-dashboard	Django 5.0.6	Read-only operational console with polled live updates.

Table 2: Service decomposition. Each runs as an isolated container with explicit CPU and memory limits.

2.3 Storage strategy

A deliberate split is maintained between the two persistence layers:

- **HDFS** holds anything large or binary: raw datasets, columnar Parquet archives, serialised `PipelineModel` directories and streaming checkpoints.
- **MySQL** holds anything small, queried transactionally or read by the web tier: registry rows with *pointers* to HDFS paths, prediction rows and counters.

No model blobs are ever written to MySQL. Storing multi-megabyte serialised models in a relational table would bloat the database, slow every registry query, and duplicate a responsibility HDFS already discharges better.

2.4 Database schema

Table	Purpose and key columns
model_registry	One row per trained model. <code>hdfs_path</code> , <code>version</code> , <code>trained_on</code> , <code>auc_score</code> , <code>accuracy</code> , <code>precision_purchase</code> , <code>recall_purchase</code> , <code>decision_threshold</code> , <code>is_active</code> . Exactly one row carries <code>is_active = TRUE</code> at any time.
predictions	One row per scored session. <code>user_session</code> , <code>event_time</code> , <code>purchase_probability</code> , <code>predicted_label</code> , <code>model_version</code> , <code>scored_at</code> . Indexed on <code>user_session</code> and <code>scored_at</code> .
running_metrics	Two-row counter table (<code>total_predictions</code> , <code>purchase_predicted</code>) incremented inside the prediction-write transaction, so dashboard reads are $O(1)$ rather than full-table counts.

Table 3: MySQL schema, created by `mysql-init/init.sql` on first container boot.

```

1/1 v + Tili: tatakae@tatakae: /mnt/master_program/semester3/big_data_essential/Projects/final_exam/bigdata-ecommerce-project
1: tatakae@tatakae: /mnt/master_program/semester3/big_data_essential/Projects/final_exam/bigdata-ecommerce-project
tatakae@tatakae: /mnt/master_program/semester3/big_data_essential/Projects/final_exam/bigdata-ecommerce-project$ docker exec -it mysql my
sql -u bigdata_user -pchangeme -e "USE model_registry; SHOW TABLES;"
mysql: [Warning] Using a password on the command line interface can be insecure.
+-----+
| Tables_in_model_registry |
+-----+
| model_registry            |
| predictions               |
| running_metrics           |
+-----+
tatakae@tatakae: /mnt/master_program/semester3/big_data_essential/Projects/final_exam/bigdata-ecommerce-project$

```

Figure 2: MySQL schema verification after first boot. The three tables are created automatically and the running counters are initialised to zero.

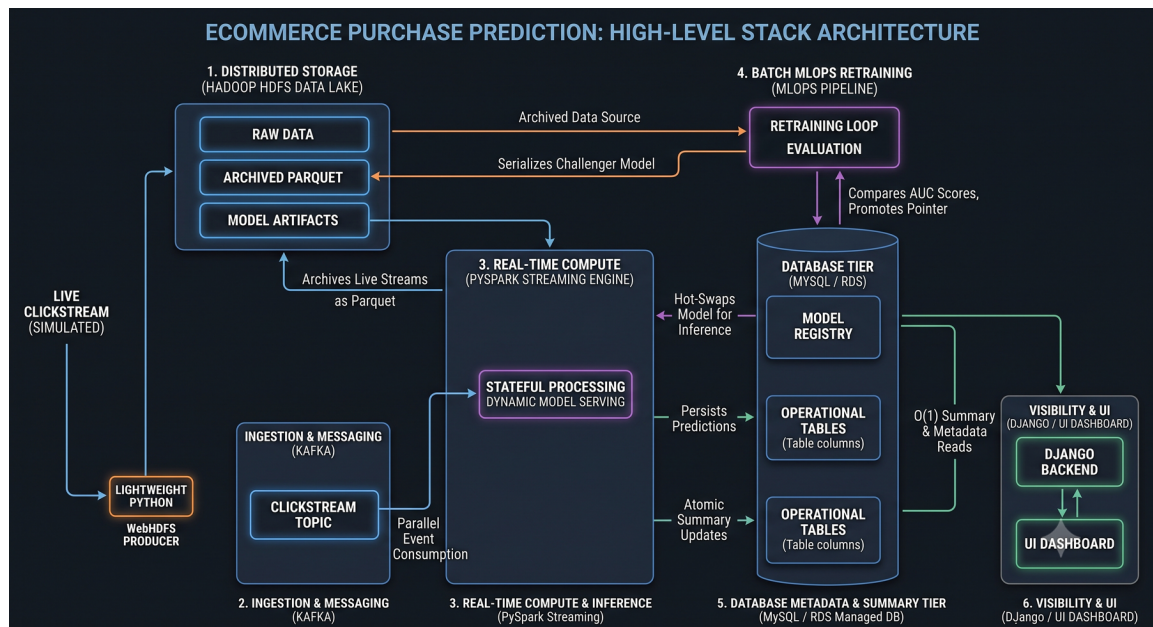


Figure 3: Multi-tier cluster overview showing all six services and the networks connecting them.

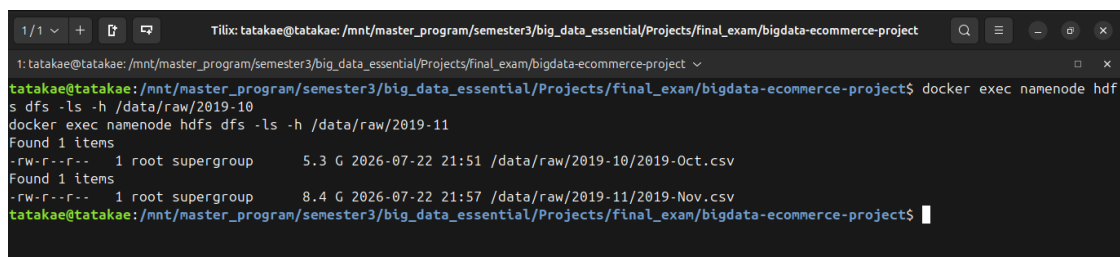
3 Implementation

3.1 Distributed storage — HDFS

The Hadoop cluster runs as a NameNode and a DataNode pair using the `bde2020` images, configured through `hadoop.env`:

```
CORE_CONF_fs_defaultFS=hdfs://namenode:9000
CORE_CONF_hadoop_http_staticuser_user=root
HDFS_CONF_dfs_replication=1
HDFS_CONF_dfs_webhdfs_enabled=true
HDFS_CONF_dfs_permissions_enabled=false
```

Replication is set to 1 because the deployment is single-node; WebHDFS is enabled because the ingestion job reads through the REST API rather than a JVM client. The raw CSVs are pushed in by `scripts/load_to_hdfs.sh`.



```
Tilix: tatakae@tatakae: /mnt/master_program/semester3/big_data_essential/Projects/final_exam/bigdata-e-commerce-project
1: tatakae@tatakae: /mnt/master_program/semester3/big_data_essential/Projects/final_exam/bigdata-e-commerce-project
tatakae@tatakae: /mnt/master_program/semester3/big_data_essential/Projects/final_exam/bigdata-e-commerce-project$ docker exec namenode hdfs
s dfs -ls -h /data/raw/2019-10
docker exec namenode hdfs dfs -ls -h /data/raw/2019-11
Found 1 items
-rw-r--r-- 1 root supergroup 5.3 G 2026-07-22 21:51 /data/raw/2019-10/2019-Oct.csv
Found 1 items
-rw-r--r-- 1 root supergroup 8.4 G 2026-07-22 21:57 /data/raw/2019-11/2019-Nov.csv
tatakae@tatakae: /mnt/master_program/semester3/big_data_essential/Projects/final_exam/bigdata-e-commerce-project$
```

Figure 4: NameNode web interface at port 9870 confirming a healthy cluster and live DataNode.

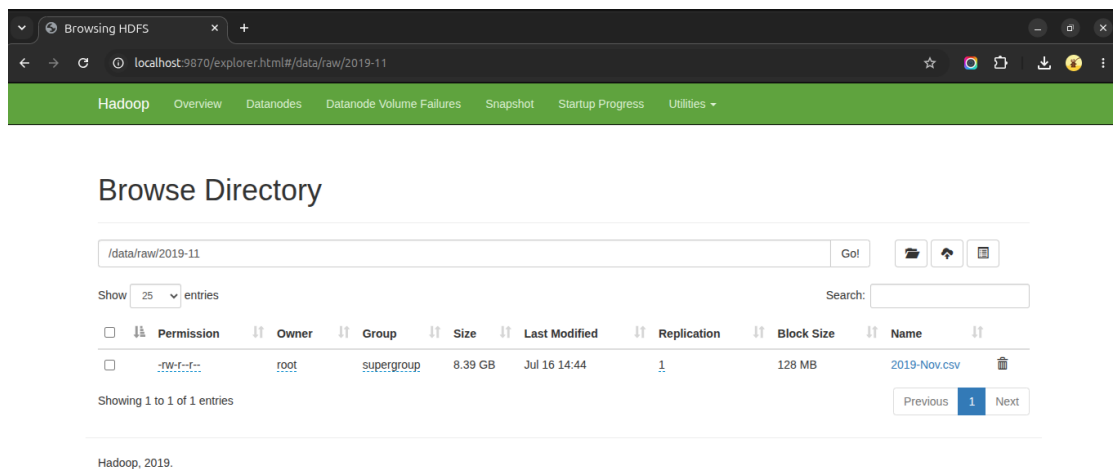


Figure 5: Raw datasets resident in HDFS under `/data/raw`, split across the DataNode.

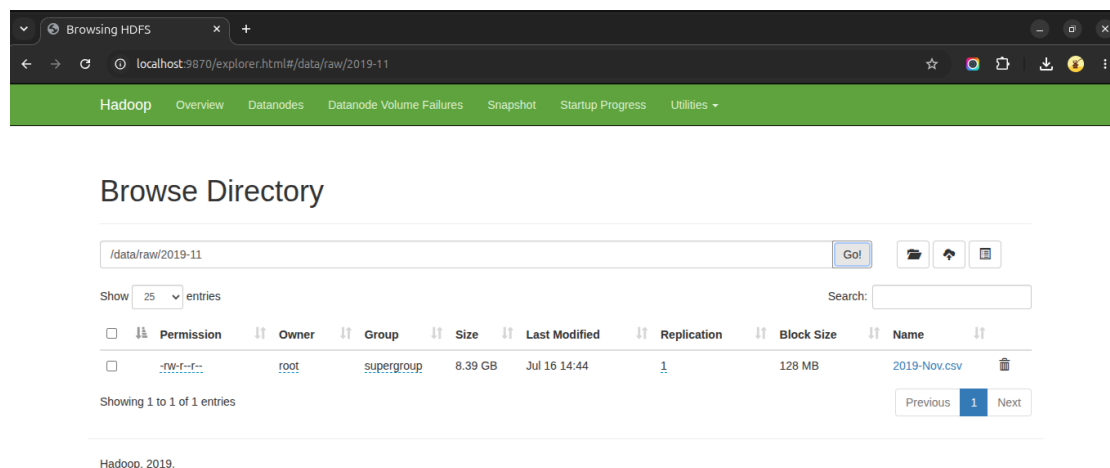


Figure 6: Block-level detail for the raw CSV, showing how HDFS has partitioned the file into distributed blocks.

3.2 Message transport — Kafka

Kafka runs in **KRaft mode**, which removes the ZooKeeper dependency entirely; the broker manages its own metadata quorum. The topic is created with three partitions:

```
kafka-topics.sh --create --topic clickstream-events \
  --partitions 3 --replication-factor 1 \
  --bootstrap-server kafka:9092
```

Every message is keyed by `user_session`. This is not decorative — Kafka routes identical keys to the same partition, so all events belonging to one session arrive at one consumer in order. Without it, session state would be scattered across partitions and per-session aggregation would be non-deterministic.

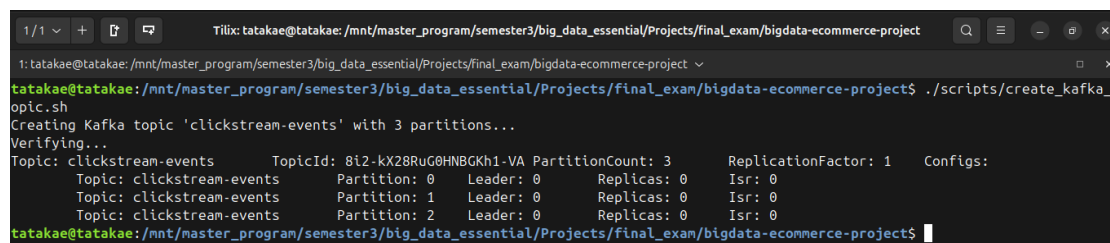


Figure 7: Kafka topic created in KRaft mode with three partitions.

3.3 Kafka producer — ingestion job

The producer deliberately does *not* use PySpark. It performs no aggregation and no machine learning; it reads rows sequentially and publishes them one at a time, which a plain Python WebHDFS client handles without carrying the memory cost of a JVM.

Two reliability problems were encountered and solved during development, and both are worth recording because they were silent failures rather than crashes:

- **The WebHDFS client has no default timeout.** A stalled connection — a redirect to an unreachable DataNode, a hung NameNode response — blocks forever with no error output. A custom `TimeoutHTTPAdapter` was mounted on the `requests` session to force every HTTP call to fail loudly after a bounded wait.

- **Kafka producer buffering.** The initial implementation using `kafka-python` hung at roughly 80,000 events. Messages were being queued into an in-memory buffer faster than the background sender thread could drain it, until the buffer filled and `produce()` itself began blocking. The fix was twofold: switch to `confluent-kafka`, whose delivery callback surfaces failures instead of retrying silently, and call `producer.poll(0)` after every send plus a periodic `flush(timeout=30)` to bound buffer growth.

```

Producer({
    "bootstrap.servers": KAFKA_BOOTSTRAP,
    "linger.ms": 50,
    "acks": "1",
    "queue.buffering.max.kbytes": 65536, # 64 MB
    "socket.timeout.ms": 30000,
    "message.timeout.ms": 60000, # fail loudly, do not retry forever
})

```

Listing 1: Producer configuration with bounded timeouts.

3.4 Kafka consumer — PySpark Structured Streaming

The streaming job subscribes to the topic and fans out into two independent queries.

Sink 1 — Parquet archive. Every parsed event is written to HDFS in Parquet format, partitioned by `event_type` and `event_date`, on a 60-second trigger. Micro-batching on a trigger interval rather than per-event avoids the small-file problem that would otherwise cripple the NameNode. This archive is what closes the retraining loop.

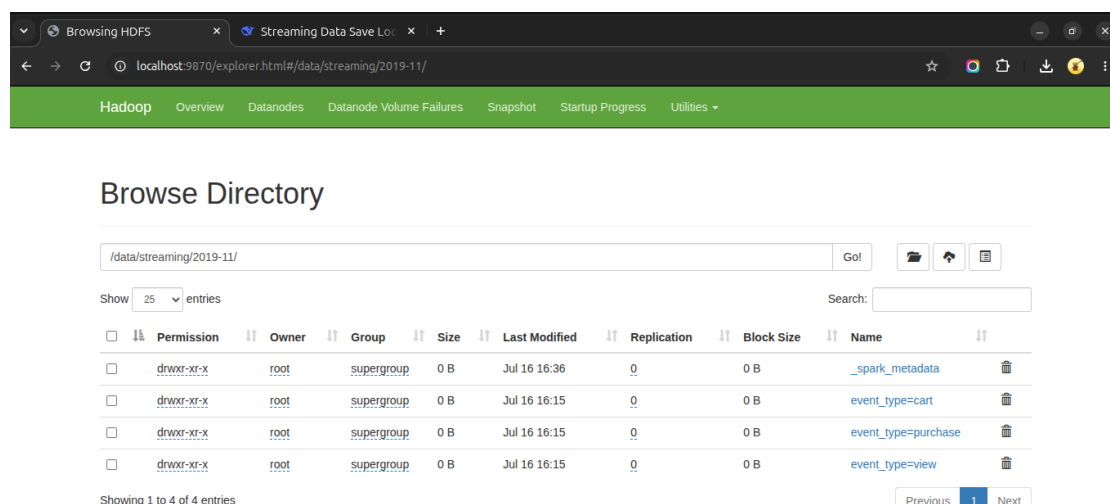


Figure 8: Streamed events archived to the HDFS Parquet data lake, partitioned by event type and date.

Sink 2 — scoring. Session aggregates are computed incrementally, scored by the active model, and written to MySQL. Three implementation choices govern its correctness:

- **foreachBatch.** MLlib's `PipelineModel.transform()` only accepts a static `DataFrame`. `foreachBatch` hands the query exactly that for each micro-batch, and is also where the MySQL write and the model hot-reload check occur.
- **Append mode over a session window.** Session windows merge as new events arrive, and a merged window cannot be retracted from a sink already written to. Append mode therefore emits exactly one scored row per session, once its 30-minute watermark closes it. The trade-off is that scoring happens at session close rather than mid-session.

- A **30-minute watermark** bounds state size and defines when a session is considered over for state-cleanup purposes, preventing unbounded memory growth in the state store.

Model hot-reload. Every micro-batch triggers a check, rate-limited to once per 30 seconds, against the registry. The model is reloaded from HDFS only if the active `hdfs_path` actually changed:

```
def refresh_champion_if_needed(spark):
    now = time.time()
    if now - _model_state["last_checked"] < MODEL_POLL_INTERVAL_SECONDS:
        return
    _model_state["last_checked"] = now

    champion = get_active_champion()
    if champion is None:
        return
    hdfs_path, version = champion
    if hdfs_path == _model_state["hdfs_path"]:
        return # unchanged, nothing to reload

    _model_state["model"] = PipelineModel.load(hdfs_path)
    _model_state["hdfs_path"] = hdfs_path
    _model_state["version"] = version
```

3.5 Feature engineering

Ten session-level features are derived, all of them behavioural aggregates over the events observed in the session window.

Feature	Definition
<code>view_count</code>	Number of <code>view</code> events
<code>cart_count</code>	Number of <code>cart</code> events
<code>remove_count</code>	Number of <code>remove_from_cart</code> events
<code>distinct_products</code>	Approximate distinct products touched
<code>distinct_categories</code>	Approximate distinct categories touched
<code>distinct_brands</code>	Approximate distinct brands touched
<code>avg_price</code>	Mean price of items interacted with
<code>max_price</code>	Highest price of items interacted with
<code>session_duration_sec</code>	Seconds between first and last non-purchase event
<code>has_cart</code>	Binary indicator that the session had any cart event

Table 4: The feature vector. `approx_count_distinct` is used for the three cardinality features, trading a small accuracy loss for tractable memory at nine-million-session scale.

3.5.1 Leakage control

The label is derived from the *full* event list — did this window ever contain a purchase? — but every feature is aggregated with purchase events excluded:

```
non_purchase = F.col("event_type") != "purchase"

.agg(
    # Label: computed across ALL events in the window
    F.max(F.when(F.col("event_type") == "purchase", 1).otherwise(0)).alias("label"),

    # Features: purchase events excluded from every one of them
    F.min(F.when(non_purchase, F.col("event_time"))).alias("session_start"),
    F.avg(F.when(non_purchase, F.col("price"))).alias("avg_price"),
    ...
```

)

This matters more than it might appear. Admitting the purchase event into feature aggregation makes purchasing sessions mechanically longer, wider and differently priced than non-purchasing ones. The model would then be reading its own target off its input, producing excellent metrics that describe nothing. An earlier iteration of this project did exactly that, and every score it produced had to be discarded.

Windows containing *only* purchase events are dropped rather than zero-imputed. Imputing zeros would teach the model that an empty session predicts a purchase — the precise inverse of the truth.

3.5.2 Train/serve feature parity

Both the training job and the streaming job group by `user_session` and a 30-minute `session_window`, and derive each feature with identical expressions. Grouping by session identifier alone would not be equivalent: in this dataset a `user_session` can persist across long idle gaps, so training would learn from one large merged session while the streaming job — which requires `session_window` for state eviction under Append mode — would score two or three smaller ones.

This is the pipeline’s most dangerous failure mode precisely because it is silent. Of the three ways the contract can drift, only one announces itself: a *missing* column raises at runtime, but a **reordered** column list, a **changed grouping key** and a **changed aggregation expression** all fail quietly. Spark scores every session against a distribution the model never saw and returns confident nonsense, with no error logged anywhere.

`tests/test_feature_parity.py` therefore parses both source files with Python’s `ast` module and compares grouping keys and per-alias aggregation expressions structurally. The tests were mutation-checked: reordering `FEATURE_COLS`, replacing the session-window grouping with a plain `groupBy("user_session")`, and dropping the purchase exclusion from a single price aggregate each produce a failure.

3.6 Prediction modelling with MLlib

3.6.1 Model selection

The classifier is chosen by the `MODEL_TYPE` environment variable rather than hardcoded, so the model actually in use is visible in configuration and in the run logs. Three options are supported, and all three accept `weightCol` in Spark 3.5, so the imbalance correction applies identically whichever is chosen:

MODEL_TYPE	MLlib class	Configuration
logistic_regression	LogisticRegression	maxIter=50 (default)
random_forest	RandomForestClassifier	numTrees=100, maxDepth=10
gbt	GBTClassifier	maxIter=50, maxDepth=5

Table 5: Available classifiers. Logistic regression was used for the results reported here.

The pipeline is a `VectorAssembler` followed by the selected classifier, persisted as a single `PipelineModel` so that the assembly step travels with the model and cannot drift from it.

3.6.2 Chronological splitting

The dataset is split 60 / 15 / 25 into train, validation and test *by session_start*, not randomly. A random split would allow the model to learn from sessions occurring after the ones it is evaluated on, which corresponds to no deployment that could ever exist. Chronological splitting is the only honest evaluation for a system whose entire purpose is to predict forward in time.

3.6.3 Class imbalance

With a 6.7% purchase rate, an unweighted model has every incentive to predict the majority class. The correction is computed from the training split at runtime rather than hardcoded, so it tracks the data as the corpus grows across retraining cycles:

```
def add_class_weights(train_df):
    counts = {row["label"]: row["count"]
              for row in train_df.groupBy("label").count().collect()}
    neg, pos = counts.get(0, 0), counts.get(1, 0)
    ratio = (neg / pos) if pos > 0 else 1.0
    return train_df.withColumn(
        "class_weight",
        F.when(F.col("label") == 1, F.lit(ratio)).otherwise(F.lit(1.0)))
```

On the October corpus this produced a ratio of **13.57**.

3.6.4 Decision threshold tuning

Because class weighting inflates predicted probabilities, the natural 0.5 cut-off is far too permissive. The trainer sweeps thresholds from 0.05 to 0.95 in steps of 0.05 on the *validation* set — never the test set — and selects the threshold maximising F_1 on the purchase class. That threshold is then set on the classifier stage before the model is persisted, so it travels with the artifact and is recorded in the registry.

3.7 Champion/Challenger retraining

Each training cycle proceeds as follows:

1. Load the October baseline from HDFS, converting CSV to Parquet on first run and reusing the Parquet thereafter.
2. If the streaming archive exists and is non-empty, deduplicate it and merge with `unionByName`. Deduplication is necessary because the Parquet archive is append-only: replaying the producer or resetting Kafka offsets writes the same events twice, and without the guard each cycle would train on an inflated, artificially reweighted copy of the same sessions.
3. Engineer session features, split chronologically, apply class weights.
4. Fit the Challenger, tune its threshold on validation, evaluate on test.
5. Load the incumbent Champion from HDFS and **re-score it on the same held-out test set**. This is what makes the comparison fair — comparing a new model's metrics against metrics recorded months ago on a different test split would be meaningless.
6. Promote only if $AUC_{\text{challenger}} - AUC_{\text{champion}} > 0.02$. Every Challenger is recorded regardless, so the registry preserves the full experimental history including rejections.

Promotion is decided on AUC rather than accuracy, for reasons Section 7 makes concrete.

3.8 Django dashboard

The dashboard is a read-only white and blue interface. Frontend JavaScript polls a `/api/stats/` JSON endpoint every eight seconds and updates Chart.js visuals in place, avoiding a full page reload.

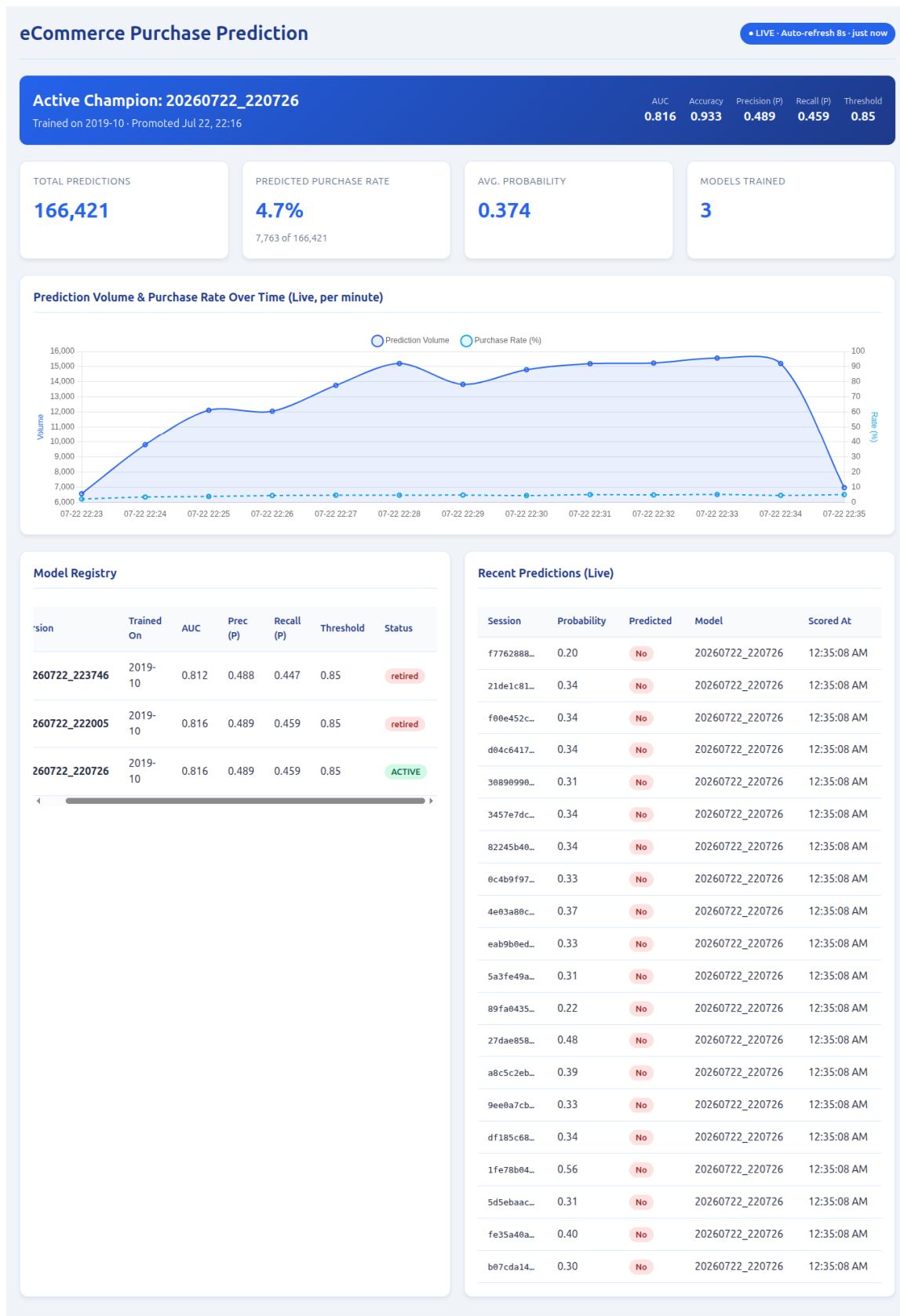


Figure 9: Live operational dashboard: headline counters, purchase-rate trend over the live stream, model registry history, and the most recent scored sessions.

Two query-design decisions keep it responsive at scale:

- **Pre-aggregated counters.** The headline figures would otherwise require `COUNT(*)` over predictions on every poll. Instead the streaming job increments the `running_metrics` table inside the *same transaction* as the insert, so a poll reads two integers rather than counting millions of rows.
- **Bounded trend query.** The per-minute chart is a `GROUP BY` restricted to a 30-minute `scored_at` window, which uses the `idx_scored_at` index instead of scanning the whole table. It also orders descending before slicing — slicing an ascending ordering returns the *oldest* buckets, which freezes the chart on the first minutes of the run and never advances.

Django's ORM models are declared `managed = False`. The tables are owned by the Spark jobs and created by `init.sql`; Django reads them and never creates, alters or drops them.

```

Tilix: tatakae@tatakae: /mnt/master_program/semester3/big_data_essential/Projects/final_exam/bigdata-ecommerce-project
... /semester3/big_data_essential/Projects/final_exam/bigdata-ecommerce-project ... /semester3/big_data_essential/Projects/final_exam/bigdata-ecommerce-project
-----
:: retrieving :: org.apache.spark#spark-submit-parent-ab2b6f39-0e8f-492f-bafd-d3dec7971ba5
confs: [default]
11 artifacts copied, 0 already retrieved (56767kB/33ms)
26/07/22 22:22:09 WARN NativeCodeLoader: Unable to load native-hadoop library for your platform... using builtin-java classes where applicable
Setting default log level to "WARN".
To adjust logging level use sc.setLogLevel(newLevel). For SparkR, use setLogLevel(newLevel).
26/07/22 22:22:25 WARN GarbageCollectionMetrics: To enable non-built-in garbage collector(s) List(G1 Concurrent GC), users should configure it(them) to spark.eventLog.gcMetrics.youngGenerationGarbageCollectors or spark.eventLog.gcMetrics.oldGenerationGarbageCollectors
26/07/22 22:22:38 WARN ResolveWriteToStream: spark.sql.adaptive.enabled is not supported in streaming DataFrames/Datasets and will be disabled.
26/07/22 22:22:39 WARN AdminClientConfig: These configurations '[key.serializer, value.serializer, enable.auto.commit, max.poll.records, auto.offset.reset]' were supplied but are not used yet.
26/07/22 22:22:40 WARN ResolveWriteToStream: spark.sql.adaptive.enabled is not supported in streaming DataFrames/Datasets and will be disabled.
[consumer.py] Both streaming queries started. Awaiting data...
26/07/22 22:22:40 WARN AdminClientConfig: These configurations '[key.serializer, value.serializer, enable.auto.commit, max.poll.records, auto.offset.reset]' were supplied but are not used yet.

[producer.py] Sent 214500 events (2036.1/sec actual, 0 skipped, 0 delivery failures)
[producer.py] Sent 215000 events (2036.8/sec actual, 0 skipped, 0 delivery failures)
[producer.py] Sent 215500 events (2037.8/sec actual, 0 skipped, 0 delivery failures)
[producer.py] Sent 216000 events (2038.6/sec actual, 0 skipped, 0 delivery failures)
[producer.py] Sent 216500 events (2039.5/sec actual, 0 skipped, 0 delivery failures)
[producer.py] Sent 217000 events (2040.1/sec actual, 0 skipped, 0 delivery failures)
[producer.py] Sent 217500 events (2041.0/sec actual, 0 skipped, 0 delivery failures)
[producer.py] Sent 218000 events (2041.9/sec actual, 0 skipped, 0 delivery failures)
[producer.py] Sent 218500 events (2042.6/sec actual, 0 skipped, 0 delivery failures)
[producer.py] Sent 219000 events (2043.6/sec actual, 0 skipped, 0 delivery failures)
[producer.py] Sent 219500 events (2044.8/sec actual, 0 skipped, 0 delivery failures)
[producer.py] Sent 220000 events (2045.5/sec actual, 0 skipped, 0 delivery failures)
[producer.py] Sent 220500 events (2046.4/sec actual, 0 skipped, 0 delivery failures)
[producer.py] Sent 221000 events (2047.5/sec actual, 0 skipped, 0 delivery failures)

```

Figure 10: Producer and consumer container logs during a live run, showing events being published to Kafka and micro-batches being scored and written to MySQL.

4 Technologies and Version Information

Layer	Technology	Version	Notes
Distributed storage	Apache Hadoop / HDFS	3.2.1	bde2020/hadoop-*:2.0.0-hadoop3.2.1-java8; replication 1; WebHDFS enabled
Message broker	Apache Kafka	3.7.0	bitnamilegacy/kafka:3.7.0; KRaft mode, no ZooKeeper; 3 partitions
Processing engine	Apache Spark (PySpark)	3.5.1	local[*] inside each job container
Scala runtime	Scala	2.12	Determines the Kafka connector artifact suffix
ML library	Spark MLlib	3.5.1	Ships with PySpark; no separate install
Database	MySQL	8.0	Local container, or AWS RDS MySQL
Web framework	Django	5.0.6	Development server; read-only dashboard
Charting	Chart.js	4.4.1	Loaded client-side from CDN
Language runtime	Python	3.10	python:3.10-slim base for all four job images
Java runtime	OpenJDK JRE headless	17	default-jre-headless; JVM side only
Orchestration	Docker Compose	v2	Single host, six services
Serialisation	Apache Parquet	—	zstd compression codec

Table 6: Complete technology stack with versions.

5 Dependencies

5.1 Spark JAR

One external JAR is required, resolved at runtime through `spark.jars.packages` and cached in a Docker volume after first download:

```
org.apache.spark:spark-sql-kafka-0-10_2.12:3.5.1
```

The coordinate encodes two version constraints that must both hold. The `_2.12` suffix must match the **Scala** version Spark was built against, and the trailing `3.5.1` must match the **PySpark** version in `requirements.txt`. A mismatch in either produces a `ClassNotFoundException` for the Kafka source at *query start* rather than at import time, which makes it considerably easier to misdiagnose than a normal import error.

5.2 Python packages by service

Service	Packages
ingestion-job	hdfs==2.7.3, confluent-kafka==2.5.0, requests==2.31.0
streaming-job	pyspark==3.5.1, pymysql==1.1.0, cryptography==42.0.8, numpy==1.26.4, pandas==2.1.1
training-job	pyspark==3.5.1, pymysql==1.1.0, cryptography==42.0.8, numpy==1.26.4, pandas==2.1.1
django-dashboard	Django==5.0.6, PyMySQL==1.1.0, cryptography==42.0.8

Table 7: Per-service Python dependencies. Each service has its own `requirements.txt` and its own image.

Two substitutions were made deliberately against the more obvious defaults:

- **confluent-kafka rather than kafka-python**, because delivery failures surface through its callback instead of being retried silently on a background thread. This directly resolved a production hang described in Section 3.3.
- **PyMySQL rather than mysqlclient**, because it is pure Python and avoids adding `libmysqlclient-dev` and a C toolchain to every image.

5.3 Resource allocation

Service	Memory limit	CPU limit
training-job	4.5 GB	2.0
streaming-job	2.0 GB	1.5
namenode	1.5 GB	1.0
kafka	1.2 GB	1.0
datanode	1.0 GB	1.0
mysql	0.75 GB	0.5
ingestion-job	0.5 GB	0.5
django-dashboard	0.5 GB	0.5

Table 8: Container resource limits. A minimum of 8 GB of free host RAM is required.

6 Testing

Two test suites run entirely offline, requiring neither a JVM, nor HDFS, nor a running MySQL instance:

```
# Cross-service contract tests
python -m pytest tests/ -v

# Dashboard tests (SQLite overlay)
cd django-dashboard && python manage.py test monitor --settings=test_settings
```

Feature parity tests. `tests/test_feature_parity.py` guards the silent-drift failure mode described in Section 3.5.2 by parsing both source files with `ast` and comparing grouping keys and aggregation expressions structurally. Parsing the source rather than importing it is what allows the test to run without PySpark installed.

Dashboard tests. `monitor/tests.py` covers the empty-database path — the frontend polls `/api/stats/` from page load, before any prediction exists, so an unguarded view would raise on the very first request — along with the pre-aggregated counter reads and the trend window bounds. A custom `test_runner.py` creates the unmanaged tables in the test database, since Django does not manage them.

7 Results and Findings

All figures in this section come from training cycle 20260726_083630: logistic regression on the October 2019 corpus, executed on 26 July 2026. This was the cold-start run — no November data had yet accumulated in the Parquet archive, so the trainer reported *“Training baseline configuration activated. Using static data lake records only”* and promoted the model unopposed as Champion v1.

```

training-job | [train.py] Best threshold: 0.85 (val F1=0.508)
training-job | [train.py] Challenger metrics: {'auc': 0.8157789606959231, 'accuracy': 0.9333487209456506, 'precision': 0.93143151373325
21, 'recall': 0.9333487209456507, 'precision_purchase': 0.4890833831309848, 'recall_purchase': 0.45874801771750423, 'decision_threshold'
: 0.85}
training-job | [train.py] Challenger model saved to /models/candidates/20260722_220726
training-job | [train.py] No existing Champion – promoting as Champion v1.
training-job | [train.py] === Cycle 20260722_220726 complete ===
training-job |
training-job | [train.py] Sleeping 2592000s until next training cycle...
```

Figure 11: Training job output: session aggregation, class weight computation, threshold sweep and Champion promotion.

7.1 Dataset after session aggregation

Quantity	Value
Sessions after windowing	9,436,444
Sessions ending in a purchase	630,085
Purchase rate	6.7%
Training split — negative class	5,291,223
Training split — positive class	389,875
Class weight ratio (neg / pos)	13.57

Table 9: Corpus characteristics after session-window aggregation.

7.2 Threshold sweep on the validation set

Threshold	Precision	Recall	F_1
0.30	0.080	0.952	0.148
0.35	0.173	0.836	0.287
0.40	0.237	0.741	0.359
0.45	0.287	0.674	0.402
0.50	0.325	0.635	0.430
0.55	0.355	0.611	0.450
0.60	0.381	0.596	0.465
0.65	0.404	0.584	0.477
0.70	0.422	0.574	0.487
0.75	0.439	0.567	0.495
0.80	0.453	0.560	0.501
0.85	0.465	0.551	0.504
0.90	0.481	0.517	0.498
0.95	0.451	0.156	0.232

Table 10: Validation threshold sweep. The selected operating point is 0.85, with $F_1 = 0.504$.

7.3 Held-out test performance

Metric	Value
AUC (ROC)	0.8280
Accuracy	0.9333
Weighted precision	0.9323
Weighted recall	0.9333
Precision, purchase class	0.4788
Recall, purchase class	0.4621
Decision threshold	0.85

Table 11: Test-set metrics for Champion v1, registered in MySQL as version 20260726_083630 with `auc_score = 0.828021` and `is_active = 1`.

7.4 Findings

Accuracy is the wrong metric here, and these numbers demonstrate exactly why. The test accuracy of 93.33% is essentially identical to what a trivial model that predicts “no purchase” for every single session would score, given a purchase rate of roughly 6.7%. The weighted precision and recall figures (0.932 and 0.933) tell the same uninformative story — they are dominated by the majority class. This is precisely why `train.py` makes promotion decisions on AUC and records `precision_purchase` and `recall_purchase` as separate columns in the registry. A submission reporting only the 93% figure would be reporting the base rate.

The model nonetheless carries substantial signal. An AUC of 0.828 means that given one purchasing and one non-purchasing session drawn at random, the model ranks the purchaser higher about 83% of the time. On the purchase class it recovers 46.2% of actual buyers at 47.9% precision. Against a 6.7% base rate, a flagged session converts roughly one time in two rather than one time in fifteen — a **seven-fold lift**. That is the commercially meaningful number, and it is entirely invisible in the accuracy figure.

The optimal threshold of 0.85 is a direct consequence of the class weighting. Weighting positive examples by 13.57 inflates predicted probabilities across the board, so the natural 0.5 cut is far too permissive: at 0.5 the model reaches recall of 0.635 but precision of only 0.325. More usefully, the sweep shows the trade-off is smooth and shallow between 0.70 and 0.90 — F_1 moves only from 0.487 to 0.498 across that entire range. The operating point can therefore be chosen on business grounds rather than statistical ones at almost no cost in aggregate performance.

That choice is genuinely open, and the right answer depends on intervention cost. At threshold 0.90 precision rises to 0.481 while recall falls to 0.517; at 0.70 precision drops to 0.422 while recall rises to 0.574. If the intervention is cheap — a banner, a free-shipping nudge — the lower threshold is correct, because a wasted impression costs nothing while a missed buyer costs margin. If the intervention is expensive — a percentage discount applied to the cart — the higher threshold is correct, because every false positive is money handed to a customer who was going to buy anyway. Below 0.35 the model degenerates into flagging almost everything: at threshold 0.30 recall is 95.2% but precision is 8.0%, barely above the base rate and operationally worthless.

The discontinuity between thresholds 0.30 and 0.35 is itself informative. Precision more than doubles, from 0.080 to 0.173, while recall falls only from 0.952 to 0.836. A jump that asymmetric implies a large, tightly clustered mass of very low-intent sessions — almost

certainly single-view bounces — which the model separates cleanly from everything else. The residual difficulty is therefore not distinguishing browsers from buyers, but distinguishing *serious* browsers from buyers, and that is where the remaining error concentrates.

The retraining loop is implemented but not yet demonstrated comparatively. Because this cycle ran before any November data had accumulated, no Challenger comparison took place. The full path — consolidating October with streamed November data via `unionByName`, re-scoring the incumbent Champion on the identical held-out test set, and promoting only on an AUC gain exceeding 0.02 — is implemented and exercised by the code, but its comparative outcome is not reported here. It is worth stating in advance that a Challenger *failing* to clear the 0.02 bar would be a legitimate finding rather than a defect: it would indicate that the October model already generalises to November traffic, which is useful information about the stability of the underlying behaviour.

7.5 Limitations

Stating these plainly is more defensible than claiming production readiness.

- **Single-node deployment.** HDFS runs with replication factor 1 on one DataNode, Kafka is a single broker, and Spark runs `local[*]` rather than against a cluster manager. The *architecture* is distributed; the *deployment* is not, and nothing here demonstrates behaviour under node failure.
- **Replayed history is not live traffic.** November 2019 events pushed at a fixed rate exhibit no diurnal pattern, no bursts, no late-arriving or out-of-order records beyond those already in the file, and no schema drift. A real stream has all four.
- **Scoring occurs at session close, not mid-session.** Append mode over a session window emits one row per session once the 30-minute watermark expires. That is correct for evaluation but limits real-time intervention — by the time a session is scored, the user has been idle for half an hour. Update mode over a fixed tumbling window would invert the trade-off, at the cost of emitting repeated partial predictions per session.
- **The feature set is deliberately shallow.** Ten aggregate behavioural features, with no product embeddings, no cross-session user history and no category-level or temporal features. Cross-session history in particular is available in the data and is the obvious next improvement.
- **Memory pressure during training.** The run log records Spark spilling cached RDD blocks to disk under the 4.5 GB container limit. Results are unaffected but the cycle is slower than necessary.

8 Deployment and Cloud Strategy

8.1 Local deployment sequence

```
cp .env.example .env

docker compose up mysql -d # schema auto-applied
docker compose up namenode datanode -d
./scripts/load_to_hdfs.sh # push CSVs into HDFS
docker compose up kafka -d
./scripts/create_kafka_topic.sh # 3 partitions

docker compose up training-job -d # ~20 min, creates Champion v1
docker logs -f training-job

docker compose up streaming-job ingestion-job django-dashboard -d
# dashboard at http://localhost:8000
```

The ordering matters. The streaming job will start before a Champion exists, but it skips scoring and logs *“no Champion loaded yet”* until one is registered.

8.2 AWS RDS migration

1. **Provision.** Launch an RDS instance using the MySQL Community Edition engine under the Free Tier. Configure the VPC security group to permit inbound TCP on port 3306 from the host’s public IP.
2. **Apply the schema.** Connect via DBeaver, MySQL Workbench or the `mysql` CLI and execute `mysql-init/init.sql`. The container’s automatic initialisation does not apply to RDS.
3. **Repoint credentials in .env:**

```
MYSQL_HOST=your-instance.abcdefgh.us-east-1.rds.amazonaws.com
MYSQL_DATABASE=model_registry
MYSQL_USER=your_rds_user
MYSQL_PASSWORD=your_rds_password
```

4. **Remove the local database.** Comment out the `mysql` service block in `docker-compose.yml` along with the corresponding `depends_on` entries, then `docker compose up -d`. This also frees roughly 750 MB of host RAM.

9 Conclusion

The system delivers a complete, working integration of the six technologies named in the course brief. Data flows from HDFS through Kafka into PySpark Structured Streaming, is scored by an MLlib model trained on historical HDFS data, persists to MySQL, and surfaces on a Django dashboard — with every stage containerised and reproducible from a single `docker compose` sequence.

Beyond the required integration, three aspects of the implementation are worth highlighting as the substantive engineering contributions:

1. **Correctness of the machine learning, not merely its presence.** Purchase events are excluded from feature aggregation, the split is chronological rather than random, and the decision threshold is tuned on a validation set held separately from the test set. Each of these is a place where a superficially working pipeline produces impressive and meaningless numbers.
2. **An enforced train/serve contract.** The feature-parity test catches the one class of bug in this architecture that fails silently, and it was verified by deliberate mutation rather than assumed to work.
3. **A closed retraining loop.** The streaming job's Parquet archive feeds the next training cycle, the registry mediates promotion, and the streaming job hot-swaps models without a restart. The loop is genuinely closed rather than described.

The clearest next steps are cross-session user history as a feature source, a comparative Challenger evaluation once sufficient November data has accumulated, and migration to a multi-node cluster where the distributed architecture is actually exercised rather than merely expressed.

A Training Run Log

Abridged output of `docker logs -f training-job` for cycle 20260726_083630.

```
[train.py] === Training cycle 20260726_083630 on 2019-10 ===
[train.py] Reusing existing Parquet at /data/processed/2019-10_parquet
[train.py] Training baseline configuration activated. Using static data lake records only.
[train.py] Sessions: 9436444, purchased: 630085 (6.7%)
[train.py] Class weight ratio (neg/pos) = 13.57 (neg=5291223, pos=389875)
[train.py] Classifier: LogisticRegression (MODEL_TYPE=logistic_regression)
[train.py] Threshold sweep on validation set:
[train.py] threshold=0.05 precision=0.064 recall=0.999 f1=0.120
[train.py] threshold=0.10 precision=0.065 recall=0.997 f1=0.121
[train.py] threshold=0.15 precision=0.065 recall=0.994 f1=0.123
[train.py] threshold=0.20 precision=0.067 recall=0.989 f1=0.126
[train.py] threshold=0.25 precision=0.070 recall=0.980 f1=0.131
[train.py] threshold=0.30 precision=0.080 recall=0.952 f1=0.148
[train.py] threshold=0.35 precision=0.173 recall=0.836 f1=0.287
[train.py] threshold=0.40 precision=0.237 recall=0.741 f1=0.359
[train.py] threshold=0.45 precision=0.287 recall=0.674 f1=0.402
[train.py] threshold=0.50 precision=0.325 recall=0.635 f1=0.430
[train.py] threshold=0.55 precision=0.355 recall=0.611 f1=0.450
[train.py] threshold=0.60 precision=0.381 recall=0.596 f1=0.465
[train.py] threshold=0.65 precision=0.404 recall=0.584 f1=0.477
[train.py] threshold=0.70 precision=0.422 recall=0.574 f1=0.487
[train.py] threshold=0.75 precision=0.439 recall=0.567 f1=0.495
[train.py] threshold=0.80 precision=0.453 recall=0.560 f1=0.501
[train.py] threshold=0.85 precision=0.465 recall=0.551 f1=0.504
[train.py] threshold=0.90 precision=0.481 recall=0.517 f1=0.498
[train.py] threshold=0.95 precision=0.451 recall=0.156 f1=0.232
[train.py] Best threshold: 0.85 (val F1=0.504)
[train.py] Challenger metrics: {'auc': 0.8280213534302987,
                             'accuracy': 0.9333386717004843,
                             'precision': 0.9322591872680215,
                             'recall': 0.9333386717004843,
                             'precision_purchase': 0.47880195097088407,
                             'recall_purchase': 0.46214341840046463,
                             'decision_threshold': 0.85}
[train.py] Challenger model saved to /models/candidates/20260726_083630
[train.py] No existing Champion - promoting as Champion v1.
[train.py] === Cycle 20260726_083630 complete ===
```

B Model Registry Verification

```
mysql> SELECT version, auc_score, decision_threshold, is_active
        FROM model_registry.model_registry;

+-----+-----+-----+-----+
| version | auc_score | decision_threshold | is_active |
+-----+-----+-----+-----+
| 20260726_083630 | 0.828021 | 0.85 | 1 |
+-----+-----+-----+-----+
```

C Source Code Repository

<https://github.com/Elthiero/auca-big-data-essential-final-exam>